循迹小车

循迹小车

- 设备连接
  - 硬件连接
  - 软件连接
- 实现思路
- 代码分析
- 实验效果
  - 注意事项

通过按键K1控制小车循迹（巡线）功能的开启和关闭，小车代码调试的环境是白色背景下循黑线前进。

> 由于前面的教程已经讲解了所有小车模块的基础知识，小车控制教程不再重复讲解，主要讲解小车功能实现思路！

设备连接

硬件连接

使用Type-B数据线连接Arduino Uno和电脑。

软件连接

打开“Arduino IDE”软件，选择开发板对应的型号和串口号。

实现思路

通过三路巡线模块获得小车相对于黑色路线的位置，并根据这个位置进行控制小车运动。

按键K1：开启或关闭小车循迹功能；

三路巡线模块：获取小车车头相对于黑色路线位置；

小车控制：

三路巡线模块位于黑色路线左边，小车左旋；

三路巡线模块位于黑色路线中间，小车直行；

三路巡线模块位于黑色路线右边，小车右旋。

> 实际代码还分析三路巡线模块全检测到黑线的情况，代码中对这种情况控制前进。

代码分析

此处只简单介绍代码内容，详细代码建议参考对应代码文件，代码文件提供在下载专区！

- 包含`Wire`、`Adafruit_PWMServoDriver`库

```
#include <Wire.h>                      // 包含Wire(I2C)通讯库 Include Wire library
#include <Adafruit_PWMServoDriver.h>  // 包含Adafruit PWMServoDriver库 Include
Adafruit PWMServoDriver library
```

- 定义电机、按键、循迹控制引脚、I2C地址、电机频率以及初始速度

```
// 定义电机控制引脚 Define motor control pins
#define Motor_L1_F_PIN 11  // 控制小车左前方电机前进 Control the motor on the left
front of the car
#define Motor_L1_B_PIN 10  // 控制小车左前方电机后退 Control the motor back on the
left front of the car
#define Motor_L2_F_PIN 8   // 控制小车左后方电机前进 Control car left rear motor
forward
#define Motor_L2_B_PIN 9   // 控制小车左后方电机后退 Control the car left rear motor
back
#define Motor_R1_F_PIN 13  // 控制小车右前方电机前进 Control the right front motor of
the car to move forward
#define Motor_R1_B_PIN 12  // 控制小车右前方电机后退 Control the motor back on the
right front of the car
#define Motor_R2_F_PIN 14  // 控制小车右后方电机前进 Control car right rear motor
forward
#define Motor_R2_B_PIN 15  // 控制小车右后方电机后退 Control car right rear motor
back

// 定义底层驱动芯片参数 Bottom-layer driver chip related parameters
#define Bottom_Layer_Driver_ADDR 0x40

// 定义PWM频率 Define PWM frequency
#define PWM_FREQUENCY 50

// 定义按键引脚和控制状态 Define pin and key(button) states
#define KEY_PIN 7
#define Press_KEY 0
#define Release_KEY 1

// 定义三路循迹模块引脚 Define the three-way line patrol module pins
#define L_TRACK_PIN A2
#define M_TRACK_PIN A1
#define R_TRACK_PIN A0

// 定义巡线阈值 Define the patrol threshold
const int Threshold = 500;

bool bCar_Switch = false;

int iCarSpeed = 50;
unsigned int uTimeOut = 0;
```

- 枚举小车状态

```
// 枚举全向小车的常见运动方式 Enumerate the common movement modes of omnidirectional
cars
enum OmniDirectionalCar {
  STOP,
```

```
  FORWARD,
  BACKWARD,
  LEFT,
  RIGHT,
  LEFT_ROTATE,
  RIGHT_ROTATE,
  LEFT_FORWARD,
  RIGHT_BACKWARD,
  RIGHT_FORWARD,
  LEFT_BACKWARD,
};
```

- 创建`Adafruit_PWMServoDriver`的实例

```
// 创建Adafruit_PWMServoDriver类的实例 Create an instance of the
Adafruit_PWMServoDriver class
Adafruit_PWMServoDriver pwm = Adafruit_PWMServoDriver(Bottom_Layer_Driver_ADDR);
```

- 设置电机速度

```
/**
 * @brief 设置单个电机速度 Setting the Motor Speed
 * @param motor_forward_pin: 控制电机前进引脚 Control the motor forward pin
 * @param motor_backward_pin: 控制电机后退引脚 Control the motor backward pin
 * @param motor_speed: 设置电机速度 Setting the Motor Speed
 * @retval 无 None
 */
void setMotorSpeed(uint16_t motor_forward_pin, uint16_t motor_backward_pin, int
motor_speed) {
  motor_speed = map(motor_speed, -255, 255, -4095, 4095);
  if (motor_speed >= 0) {
    pwm.setPWM(motor_forward_pin, 0, motor_speed);
    pwm.setPWM(motor_backward_pin, 0, 0);
  } else if (motor_speed < 0) {
    pwm.setPWM(motor_forward_pin, 0, 0);
    pwm.setPWM(motor_backward_pin, 0, -(motor_speed));
  }
}
```

- 设置小车状态

```
/**
 * @brief 设置小车运动方式和速度 Set the car movement mode and speed
 * @param Movement: 小车运动方式 Car movement
 * @param Speed: 小车运动速度 Car speed
 * @retval 无 None
 */
void setCarMove(uint8_t Movement, int Speed) {
  switch (Movement) {
    case STOP:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
```

```
    break;
  case FORWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case BACKWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  case LEFT:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  case RIGHT:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case LEFT_ROTATE:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case RIGHT_ROTATE:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
    break;
  case LEFT_FORWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
    break;
  case RIGHT_BACKWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, -Speed);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, -Speed);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
    break;
  case RIGHT_FORWARD:
    setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, Speed);
    setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
    setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
    setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, Speed);
    break;
  case LEFT_BACKWARD:
```

```
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, -Speed);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, -Speed);
      break;
    default:
      setMotorSpeed(Motor_L1_F_PIN, Motor_L1_B_PIN, 0);
      setMotorSpeed(Motor_L2_F_PIN, Motor_L2_B_PIN, 0);
      setMotorSpeed(Motor_R1_F_PIN, Motor_R1_B_PIN, 0);
      setMotorSpeed(Motor_R2_F_PIN, Motor_R2_B_PIN, 0);
      break;
  }
}
```

- 获取按键状态

```
/**
 * @brief 获取按键状态 Get key(button) status
 * @param pin: 按键控制引脚 Control key(button) pins
 * @retval 按键状态 Key(button) Status
 */
int getKeyState(uint8_t pin) {
  if (digitalRead(pin) == LOW) {
    delay(20);
    if (digitalRead(pin) == LOW) {
      while (digitalRead(pin) == LOW)
        ;
      return Press_KEY;
    }
    return Release_KEY;
  } else {
    return Release_KEY;
  }
}
```

- 小车循迹功能

```
/**
 * @brief 小车巡线功能 Car line patrol function
 * @param iLeftPin: 左边循迹传感器 Left tracking sensor
 * @param iMidPin: 中间循迹传感器 Middle tracking sensor
 * @param iRightPin: 右边循迹传感器 Right tracking sensor
 * @retval 无 None
 */
void PatrolCar(int iLeftPin, int iMidPin, int iRightPin) {
  int leftValue = analogRead(iLeftPin);
  int middleValue = analogRead(iMidPin);
  int rightValue = analogRead(iRightPin);

  // 进行简单的方向判断打印 The following is a simple judgment
  if ((leftValue < Threshold && middleValue > Threshold && rightValue <
Threshold) | (leftValue > Threshold && middleValue > Threshold && rightValue >
Threshold)) {
    setCarMove(FORWARD, iCarSpeed);
```

```
  } else if (leftValue > Threshold && middleValue < Threshold && rightValue <
Threshold) {
    setCarMove(LEFT_ROTATE, iCarSpeed);
  } else if (leftValue < Threshold && middleValue < Threshold && rightValue >
Threshold) {
    setCarMove(RIGHT_ROTATE, iCarSpeed);
  } else if (leftValue < Threshold && middleValue < Threshold && rightValue <
Threshold) {
    uTimeOut++;
    delay(20);
    if (uTimeOut > 100) {
      uTimeOut = 0;
      setCarMove(STOP, 0);
    }
  }
}
```

- 设置小车功能开关

```
/**
 * @brief 设置小车功能开关 Set the car function switch
 * @param 无 None
 * @retval 开启/关闭 true/false
 */
bool setCarSwitch() {
  if (getKeyState(KEY_PIN) == Press_KEY) {
    bCar_Switch = !bCar_Switch;
  }
  return bCar_Switch;
}
```

- 初始化代码

```
void setup() {
  Wire.begin();                    // 初始化I2C通讯 Initialize I2C communication
  delay(1000);                     // 如果小车功能异常，可以增加这个延时 If the function
is abnormal, you can increase the delay
  pwm.begin();                     // PWM初始化 Initialize the Pulse Width
Modulation (PWM) library
  pwm.setPWMFreq(PWM_FREQUENCY);  // 设置PWM频率 Set the PWM frequency
  setCarMove(STOP, 0);             // 设置小车停止状态 Set the car to stop state
}
```

- 循环运行代码

```
void loop() {
  // 按键控制小车循迹功能启停 The key control car tracking function start and stop
  if (setCarSwitch()) {
    PatrolCar(L_TRACK_PIN, M_TRACK_PIN, R_TRACK_PIN);
  } else {
    setCarMove(STOP, 0);
  }
}
```

实验效果

编译程序成功后，上传代码到Arduino Uno开发板。

程序启动后，按下K1按键，可以启动和关闭小车循迹（巡线）功能。

实际使用：我们可以将小车放到指定的赛道路线上进行测试（赛道选择白色背景黑色路线的赛道），按下K1按键启动小车功能，然后可以观察整个巡线效果，若需要停止巡线功能，可以按下K1按键关闭小车巡线功能。

> 驱动电机需要外接电池组并且打开拓展板开关才可以正常驱动。
> 烧录程序不可以使用别的程序占用串口或者外接串口通讯模块（例如：WiFi摄像头模块），不然无法烧录程序或者提示错误信息！

注意事项

若小车巡线过程中抖动比较严重，可以尝试修改小车速度和三路巡线模块判断阈值。

修改速度：直接在小车代码进行电机速度修改；

修改阈值：将三路巡线模块放在黑色背景和白色背景下，分别记录两种环境下的阈值，一般程序中的阈值取两者的中值，用户可以自行修改和调试。

> 我们可以给小车烧录【1.Car peripherals foundation：09.Track_Print.ino】的代码，串口会打印三路巡线模块接口数据和小车需要变化的方向，我们只需要关注三路巡线模块接口数据。

小车环境：黑色赛道的宽度约1.5cm比较合适，为了避免红外光对传感器产生干扰，我们需要在室内使用该模块。